

Classifying Music Genres from Audio Features

Justyna Palikowska, Maria Yui Gonzalez
Applied ML in Python – LMU
J.Palikowska@campus.lmu.de, M.Yui@campus.lmu.de

July 16, 2025

1 Task Overview

The goal of this project is to classify music genres from audio using machine learning methods. We used the GTZAN dataset [5], which contains 1000 audio clips (each 30 seconds long), evenly distributed across 10 genres. The dataset includes 1000 spectrograms and two csv files of numerical features (MFCCs, chroma, spectral contrast, etc.): one with 1000 records (30-second clips) and another with 10,000 records (each clip divided into ten 3-second segments). We chose to work with the 30-second version for time efficiency.

Music genre classification is a challenging task due to overlapping acoustic characteristics across genres and the generally low accuracy, even for human listeners (around 70%)[1]. Additionally, the GTZAN dataset has known issues such as duplicates and labeling errors[6], but we decided to use it as it is since it remains widely adopted in research. Another challenge was understanding how the spectrograms and audio features actually represent the music.

2 Methods

For this project we trained and compared two supervised models on MFCC features: a classical Support Vector Machine (binary SVM with an RBF kernel using both One vs. Rest and One vs. One classification strategies) and a neural Multilayer Perceptron (MLP). As an extension, we also trained a Convolutional Neural Network (CNN) on mel spectrograms.

2.1 Classical model: Kernel SVM

We choose Kernel SVM model. We decided this model over the Logistic Regression and the linear SVM model because they perform better on linearly separable data and for the task of classifying music genres it does not look to be the case. Moreover, the Logistic Regression model supposes independent observations and a linearity relationship between the independent variables [3].

The next step was to select a specific kernel and we agreed that trying a Gaussian kernel $K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$ would be the best first approach because it is used mainly when the shape of the data is unknown. After that, we needed to calibrate the parameter σ , also called bandwidth. First, we used $\sigma = 1$ and, combined with a lack of normalization, the accuracy was really bad (around 10%). To fix the problem, we used the median heuristic [2] and a z-score normalization. The median heuristic method $\sigma = \sqrt{\text{median}(\|x_i - x_j\|^2)/2}$ comes from the approach that the bandwidth and the family of pairwise distances would need to have the same order and picking the bandwidth to be the median of that family of distances.

2.2 First neural model: MLP

The first implementation used an MLP model with the Gradient Descent optimizer, which resulted in very low training and test accuracy. As a second attempt, we implemented our own version of the Adam optimizer, which typically performs better for deep models due to its combination of momentum and adaptive learning rate [4]. This turned out to be a good choice.

We used ReLU as the activation function, softmax for the output layer, and Cross-Entropy as the loss function. This combination was also introduced in the lecture and is known to yield strong results for multiclass classification problems. Additionally, we defined a helper function to automatically and safely convert numeric labels into one-hot encoded vectors.

2.3 Second neural model: CNN

We chose a Convolutional Neural Network (CNN) for spectrograms, as this architecture is well-suited for image-like data. We implemented a custom `SimpleCNN` using the PyTorch library, since writing the implementation from scratch would be too complex. The spectrograms were first transformed to match the expected input format. The model consists of three convolutional layers with increasing channel sizes (32, 64, 128), each followed by ReLU activation and max pooling, then a flattening layer and two fully connected layers with dropout for regularization. It was trained on mel spectrograms using the Adam optimizer (learning rate = 0.001) and Cross-Entropy loss. Training accuracy and loss were recorded after each epoch for analysis.

The chosen architecture served as a standard baseline. We experimented with various dropout values; 0.0 outperformed slightly 0.5 and was therefore used. We tried applying separate transformation pipelines: a simple one to the test data (resizing, normalization) and a more complex one to the train data (resizing, random horizontal flips, affine transformations), but it led to only minor improvements in accuracy.

3 Experiments and Results

The main results are presented in the following way: a table 1 comparing the test accuracies, a figure 1 comparing the confusion matrices of the first three models (two Kernel SVM and one MLP) and another figure 2 comparing the confusion matrix of the CNN models and showing the learning curve of the CNN model with 100 epochs. We also included F1-scores and accuracies in table 2 for every model based on the confusion matrix using the function `Classification report` from `Sklearn`. Details about that function are included in the section `Evaluation Methods` in the notebook. We chose to display the F1-scores because it combines precision and recall making it a balanced metric. CNN 25 refers to the CNN model trained with 25 epochs and the same for CNN 100.

Overall Accuracy				
OvR SVM	OvO SVM	MLP	CNN 25	CNN 100
68.5%	71%	74.5%	58%	64.5%

Table 1: Test accuracy for all models

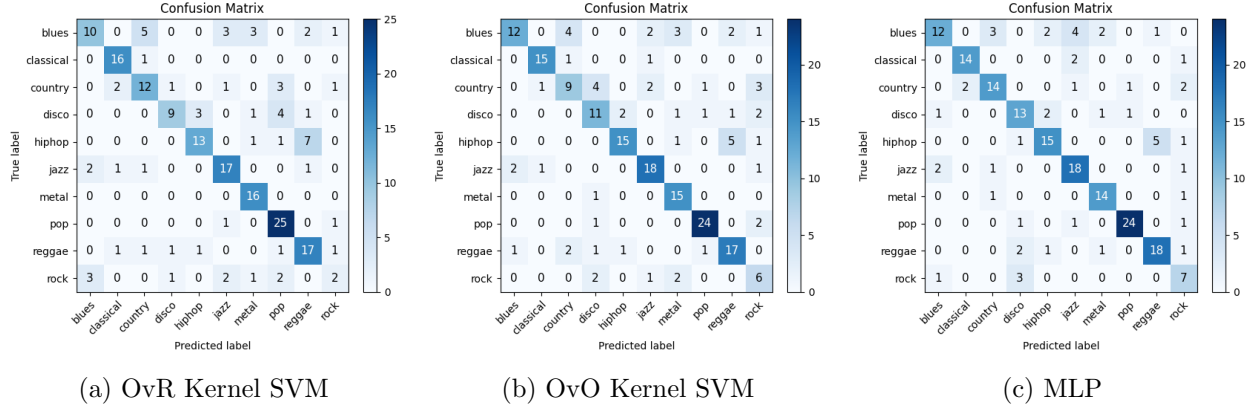


Figure 1: Confusion matrix comparison of the first two models

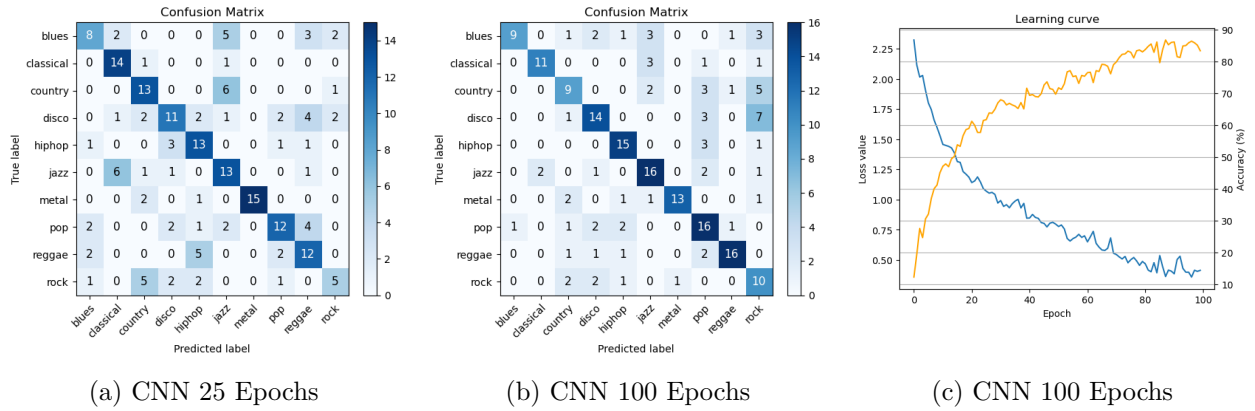


Figure 2: CNN model main results

4 Discussion

The performance of the first three models trained on MFCC features is satisfactory. According to [1], models based on numerical features rarely exceed human accuracy (around 70 %), so our results meet expectations. The overall test accuracy is similar across models, with some variation across specific genres. We also compared two multiclass strategies: One-vs-Rest and One-vs-One, which in our case gave comparable results.

The CNN trained on spectrograms achieved slightly disappointing accuracy. As a more complex model, we expected it to outperform SVM and MLP. Due to time constraints, we were not able to fully tune the model. Although training accuracy improved between the 25-epoch and 100-epoch versions, the test accuracy increase was minor. Genre-specific performance varied, and analysis of the learning curve suggests that the dataset might have been too small and resulted in overfitting. Additionally, known issues in the GTZAN dataset [6] may have impacted the outcome.

Table 2 provides a clear comparison of performance across genres and models. It shows that rock is the most difficult genre to classify, while pop, classical, and metal show the top performance.

Future work could involve tuning the CNN architecture, exploring alternative features, or using a different dataset. As suggested in [1], incorporating insights from human perception and auditory neurophysiology may also enhance performance. However, reaching 100 % accuracy is likely unrealistic, as genre classification can be subjective and even humans are not always consistent.

F1-scores					
Class	OvR SVM	OvO SVM	MLP	CNN 25	CNN 100
blues	0.51	0.62	0.60	0.47	0.60
classical	0.86	0.88	0.85	0.72	0.76
country	0.60	0.50	0.72	0.59	0.49
disco	0.60	0.58	0.68	0.50	0.60
hiphop	0.67	0.75	0.71	0.60	0.75
jazz	0.74	0.78	0.75	0.52	0.68
metal	0.84	0.79	0.82	0.81	0.81
pop	0.79	0.89	0.91	0.59	0.60
reggae	0.67	0.71	0.77	0.52	0.80
rock	0.24	0.44	0.54	0.38	0.44
Accuracy	0.69	0.71	0.74	0.58	0.65

Table 2: F1-scores for every model

References

- [1] Mingwen Dong. “Convolutional Neural Network Achieves Human-level Accuracy in Music Genre Classification”. In: *CoRR* abs/1802.09697 (2018). arXiv: 1802.09697. URL: <http://arxiv.org/abs/1802.09697>.
- [2] Damien Garreau, Wittawat Jitkrittum, and Motonobu Kanagawa. *Large sample analysis of the median heuristic*. 2018. arXiv: 1707.07269 [math.ST]. URL: <https://arxiv.org/abs/1707.07269>.
- [3] GeeksforGeeks. *Logistic Regression in Machine Learning*. URL: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>.
- [4] GeeksforGeeks. *What is Adam Optimizer?* URL: <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>.
- [5] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. <https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>. 2021.
- [6] Bob L. Sturm. “The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use”. In: *Journal of New Music Research* (2014). URL: <https://arxiv.org/abs/1306.1461>.